East West University

Department of CSE

Course Name: CSE207 DATA STRUCTURE	
Course Code: CSE207 **Section:** 07	
Project Name: Employee Management System	
Semester and Year : Fall-2023	**Group No :06**
Name of Students: MD. Shahrukh Hossain Shihab Maisha Rahman Swarna Rani Dey Sabira Musfirat Suhita Rakib Hasan Ovi **Student Id:** 2022-1-60-372 2022-1-60-371 2022-1-60-340 2022-1-60-206 2022-1-60-342	**Course Instructor's Information:** Md. Manowarul Islam Department of Computer Science & Engineering
Date of Report Submission : 27/12/2023	

ABSTRACT

The Employee Management System project presents a solution for effectively organizing and managing employee information within an organizational context. This report provides an in-depth exploration of the project, detailing the conceptualization, implementation, and utilization of key data structures.

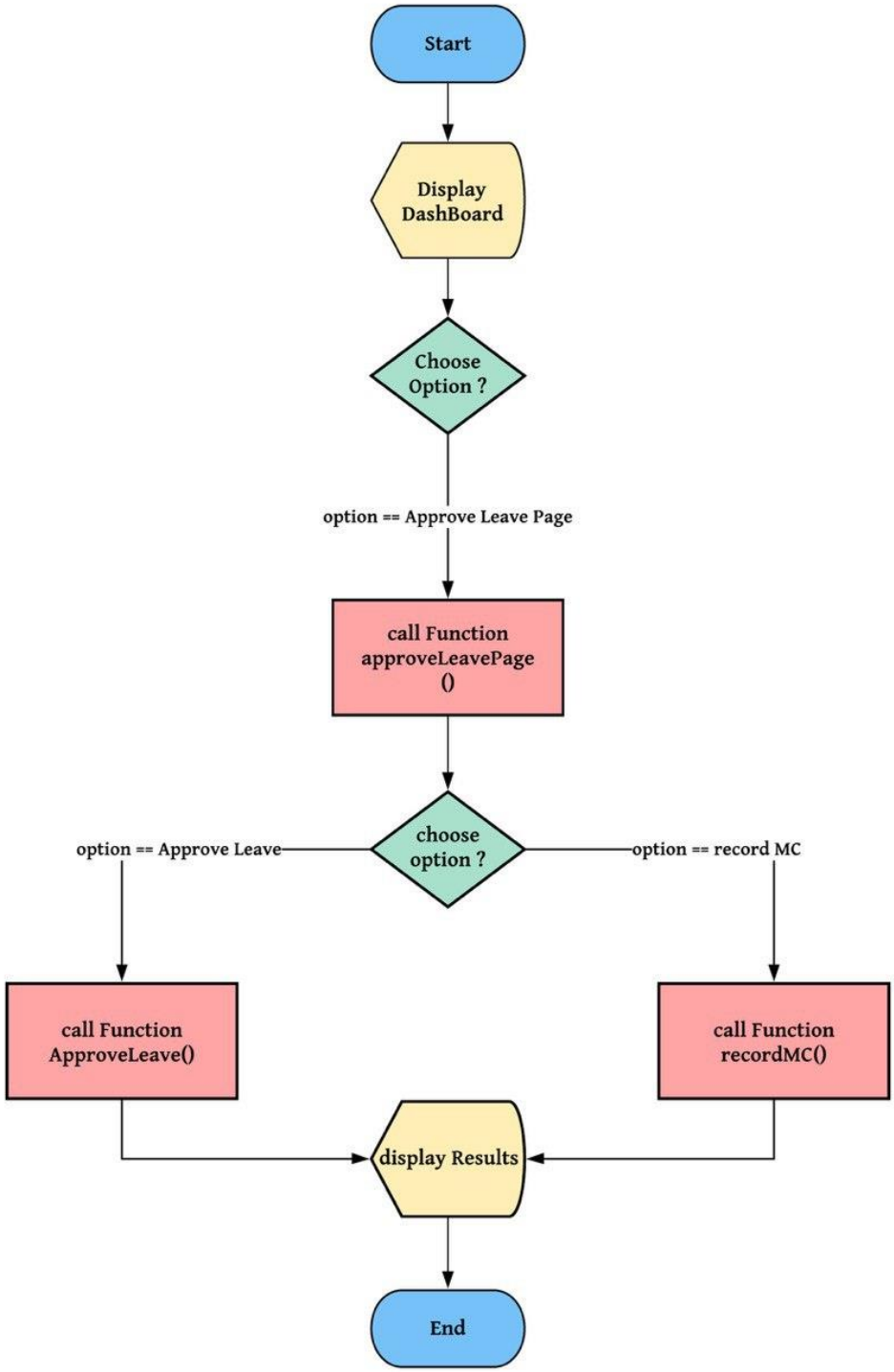
DISCUSSION

1. **Dynamic Memory Allocation:**
    - **Usage:** Dynamic memory allocation is crucial for creating and managing a variable number of employee records.
    - **Explanation:** The **malloc** function is used to dynamically allocate memory for each new employee record. This property enables the system to handle a flexible number of employees based on the organizational requirements.
2. **Insertion at the Beginning of the List:**
    - **Usage:** New employee records are inserted at the beginning of the linked list.
    - **Explanation:** The **insert** function adds a new employee at the front of the linked list. This property ensures constant time complexity for insertion, making the system efficient for frequent additions.
3. **Deletion by Value (Employee Number):**
    - **Usage:** Employees are deleted based on their unique employee number.
    - **Explanation:** The **deleteNode** function removes an employee from the linked list based on the provided employee number. This property allows for efficient deletion of specific records.
4. **Traversal of the Linked List:**
    - **Usage:** To search for and display employee information.

- **Explanation:** The **search** and **display** functions traverse the linked list, allowing efficient access to employee data. This property ensures that the system can locate and present employee details accurately.

5. **Flexible Storage of Employee Attributes:**
    - **Usage:** Storage of various attributes such as employee number, name, age, designation, and salary.
    - **Explanation:** The **Employee** structure incorporates different data types to store diverse employee attributes. This property facilitates the storage of varied information associated with each employee.
6. **Memory Deallocation:**
    - **Usage:** Ensuring proper release of memory after employee deletion.
    - **Explanation:** The **free** function is used to deallocate memory occupied by an employee record when it is deleted. This property prevents memory leaks and contributes to the overall robustness of the system.
7. **User Interaction through Menu-Driven Interface:**
    - **Usage:** Providing a user-friendly interface for interaction.
    - **Explanation:** The system employs a menu-driven interface to allow users to perform various operations, emphasizing a straightforward and intuitive approach to managing employee data.

FLOWCHART


Start
Display DashBoard
Choose Option ?
option == Approve Leave Page
call Function approveLeavePage ()
choose option ?
option == Approve Leave
option == record MC
call Function ApproveLeave()
call Function recordMC()
display Results
End

ALGORITHM

- **Step 1:** Start
- **Step 2:** Declare variables employeesList, choice, id, years, wage, fullName[MAX_NAME], role[MAX_DESIGNATION].
- **Step 3:** Print a welcome message and menu options for the Employee Management System.
- **Step 4:** Enter into an infinite loop (while loop) for user interaction.
- **Step 5:** Prompt the user to enter an option (1 to 5).
- **Step 6:** Use a switch statement to perform the following based on the user's choice:
    - Case 1: - Prompt the user to enter Employee ID, full name, years of service, role, and wage. - Call the addEmployee function to add a new employee to the employeesList.
    - Case 2: - Prompt the user to enter the employee ID to be removed. - Call the removeEmployee function to remove the specified employee from the employeesList.
    - Case 3: - Prompt the user to enter the employee ID to be found. - Call the findEmployee function to display the details of the specified employee
    - Case 4: - Check if the employeesList is empty. - If not empty, call the displayEmployees function to display details of all employees. - If empty, print a message indicating that the list is empty.
    - Case 5: - Print an exit message.
    - Exit the program using the exit(0) function.
- **Step 7:** Repeat the loop until the user chooses to exit.
- **Step 8:** Stop

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_DESIGNATION 50
#define MAX_NAME 50

struct Employee {
    int empID;
    char empFullName[MAX_NAME];
    int empYears;
    int empWage;
    char empRole[MAX_DESIGNATION];
    struct Employee* next;
};

// insert employee
struct Employee* addEmployee(struct Employee* head, int id, int years, int wage, char
fullName[], char role[]) {
    struct Employee* newEmployee = (struct Employee*)malloc(sizeof(struct Employee));
    if (newEmployee == NULL) {
        printf("\n Allocation failed \n");
        exit(0);
    }
```

```
    newEmployee->empID = id;
    newEmployee->empYears = years;
    newEmployee->empWage = wage;
    strncpy(newEmployee->empFullName, fullName, MAX_NAME - 1);
    strncpy(newEmployee->empRole, role, MAX_DESIGNATION - 1);
    newEmployee->empFullName[MAX_NAME - 1] = '\0';
    newEmployee->next = head;
    return newEmployee;
}

// delete employee by ID
struct Employee* removeEmployee(struct Employee* head, int id) {
    struct Employee* current = head;
    struct Employee* previous = NULL;

    while (current != NULL) {
        if (current->empID == id) {
            if (previous != NULL) {
                previous->next = current->next;
            } else {
                head = current->next;
            }
            free(current);
            printf("\n Removed\n");
            return head;
        }
```

```
        previous = current;
        current = current->next;
    }

    printf("\n Employee ID %d not found\n", id);
    return head;
}

// search an employee by ID
void findEmployee(struct Employee* head, int ID) {
    struct Employee* current = head;

    while (current != NULL) {
        if (current->empID == ID) {
            printf("\n ID found:\n");
            printf("\n Employee ID: %d", current->empID);
            printf("\n Name: %s", current->empFullName);
            printf("\n Years: %d", current->empYears);
            printf("\n Role: %s", current->empRole);
            printf("\n Wage: %d\n", current->empWage);
            return;
        }
        current = current->next;
    }

    printf("\n Employee ID %d not found\n", ID);
}
```

```
// display all employees
void displayEmployees(struct Employee* head) {
    struct Employee* current = head;

    while (current != NULL) {
        printf("\n Employee ID: %d", current->empID);
        printf(" \n Name: %s", current->empFullName);
        printf(" \n Years: %d", current->empYears);
        printf("\n Role: %s", current->empRole);
        printf("\n Wage: %d\n", current->empWage);
        current = current->next;
    }
}

int main() {
    struct Employee* employeesList = NULL;
    int choice;
    int id;
    int years;
    int wage;
    char fullName[MAX_NAME];
    char role[MAX_DESIGNATION];

    printf("\t\t\t------------------WELCOME TO EMPLOYEE MANAGEMENT SYSTEM----------
-------\n\n");
```

```
printf("\t\t\t\t* Press 1 to Add an Employee \n");
printf("\t\t\t\t* Press 2 to Remove an Employee \n");
printf("\t\t\t\t* Press 3 to Find an Employee \n");
printf("\t\t\t\t* Press 4 to Display Employee Data \n");
printf("\t\t\t\t* Press 5 to EXIT \n");

while (1) {
    printf("Enter option: \n");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("\n Enter the Employee ID: ");
            scanf("%d", &id);
            printf(" Enter the Employee full name: ");
            scanf("%s", fullName);
            printf(" Enter the Employee years of service: ");
            scanf("%d", &years);
            printf(" Enter the Employee role: ");
            scanf("%s", role);
            printf(" Enter the Employee wage: ");
            scanf("%d", &wage);
            employeesList = addEmployee(employeesList, id, years, wage, fullName, role);
            break;
        case 2:
            printf("\n Enter the employee ID to be removed: ");
            scanf("%d", &id);
```

```
            employeesList = removeEmployee(employeesList, id);
            break;
        case 3:
            printf("\n Enter the employee ID to be found: ");
            scanf("%d", &id);
            findEmployee(employeesList, id);
            break;
        case 4:
            if (employeesList == NULL) {
                printf("\n List is empty.");
            } else {
                displayEmployees(employeesList);
            }
            break;
        case 5:
             printf("Exited successfully.\n");
            return 0;
        default:
            printf("Invalid option. Please try again.\n");
            break;
      }
   }
}
```

OUTPUT

DISPLAY:

```
------------------WELCOME TO EMPLOYEE MANAGEMENT SYSTEM-----------------

        * Press 1 to Add an Employee
        * Press 2 to Remove an Employee
        * Press 3 to Find an Employee
        * Press 4 to Display Employee Data
        * Press 5 to EXIT
```

ADD AN EMPLOYEE:

```
Enter option:
1

 Enter the Employee ID: 11
 Enter the Employee full name: rakib
 Enter the Employee years of service: 4
 Enter the Employee role: manager
 Enter the Employee wage: 54000
Enter option:
1

 Enter the Employee ID: 12
 Enter the Employee full name: rabin
 Enter the Employee years of service: 6
 Enter the Employee role: assistant
 Enter the Employee wage: 35000
Enter option:
```

DELETE AN EMPLOYEE:

```
Enter option:
2

 Enter the employee ID to be removed: 12

 Removed
```

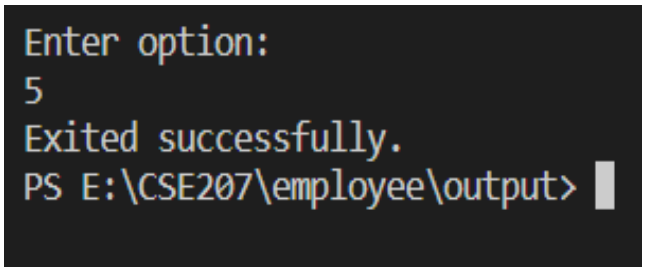
FIND AN EMPLOYEE:

```
Enter option:
3

 Enter the employee ID to be found: 11

 ID found:

 Employee ID: 11
 Name: rakib
 Years: 4
 Role: manager
 Wage: 54000
Enter option:
```

VIEW ALL EMPLOYEE:

```
Enter option:
4

 Employee ID: 11
 Name: rakib
 Years: 4
 Role: manager
 Wage: 54000
Enter option:
```

EXIT:



CONCLUSION

The Employee Management System project streamlines HR operations, enhancing efficiency by automating tasks like payroll, attendance tracking, and employee records. Its user-friendly interface simplifies access to data, fostering better decision-making. The system's robustness ensures data security and confidentiality, empowering organizations to effectively manage their workforce. Its implementation promises improved organizational productivity, reduced manual workload, and better resource allocation. Overall, the Employee Management System stands as a pivotal solution for modern businesses seeking streamlined HR processes and enhanced operational control.